



Module M26

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

Type Casting

Comparison

Built-in Type

Promotion &
Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

Programming in Modern C++

Module M26: Polymorphism: Part 1: Type Casting

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Understood Hierarchy or ISA Relationship in OOAD and introduced the Semantics of Inheritance in C++
- Discussed the effect of inheritance on Data Members and Object Layout; and on Member Functions with special reference to Overriding and Overloading
- Understood the need and use of `protected` Access specifier
- Discussed the Construction and Destruction process of class hierarchy and related Object Lifetime
- Using the Phone Hierarchy as an example analyzed the design process with inheritance
- Introduced restricted forms of inheritance and `protected` specifier
- Discussed how `private` inheritance is used for *Implemented-in-terms-of* Semantics



Module Objectives

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Understand type casting and the difference between implicit and explicit casting
- Understand type casting in a class hierarchy
- Understand the notions of upcast and downcast



Module Outline

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

1 Weekly Recap

2 Type Casting

- Basic Notions
- Comparison of Implicit and Explicit Casting
- Built-in Type
 - Promotion & Demotion
- Unrelated Classes
- Inheritance Hierarchy
 - Upcast
 - Downcast

3 Module Summary



Type Casting

Module M26

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

Type Casting

Comparison

Built-in Type

Promotion &
Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

NPTEL

Type Casting



Type Casting: Basic Notions

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Casting is performed when a *value (variable) of one type* is used in place of *some other type*. Converting an expression of a given type into another type is known as **type-casting**

```
int i = 3;
double d = 2.5;
```

```
double result = d / i; // i is cast to double and used
```

- Casting can be **implicit** or **explicit**

```
d = i;           // implicit: int to double
i = d;           // implicit: warning: '=' : conversion from 'double' to 'int': possible loss of data
d = (double)i;   // explicit: int to double
i = (int)d;      // explicit: double to int
```

- Casting Rules can be grossly classified for:
 - Built-in types
 - Unrelated types
 - Inheritance hierarchy (static)
 - Inheritance hierarchy (dynamic)



Comparison of Implicit and Explicit Casting

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

Implicit Casting

- Done *automatically*
- *No data loss*, for promotion
Compiler will be *silent*
- *Possible data loss*, for demotion
Compiler will issue *warning*
- *No throwing of exception* – is *type safe*
- Requires *no special syntax*
- *Avoid, if possible*
- *Possible only* in *static* time
- *May be disallowed* for *User-Defined Types*, but *cannot be disallowed* for *built-in types*

Explicit Casting

- Done *programmatically*
- Data loss *may or may not take place*
Compiler will be *silent*
- *May throw* for wrong type casting
- Requires *cast operator* for conversion
C style operator: (*< type >*)
C++ style operators:
`const_cast,`
`static_cast,`
`dynamic_cast,` and
`reinterpret_cast`
- *Avoid C style cast*
Use C++ style cast
- *Possible* in *static* as well as *dynamic* time
- *May be defined* for *User-Defined Types*



Type Casting Rules: Built-in Type

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Various type castings are possible between built-in types

```
int i = 3;
double d = 2.5;
```

```
double result = d / i; // i is cast to double and used
```

- Casting rules are defined between numerical types, between numerical types and pointers, and between pointers to different numerical types and `void`
- Casting can be **implicit** or **explicit**

```
int i = 3;
double d = 2.5, *p = &d;
```

```
d = i;           // implicit: int to double
i = d;           // implicit: warning: '=' : conversion from 'double' to 'int': possible loss of data
```

```
d = (double)i;  // explicit: int to double
i = (int)d;     // explicit: double to int
```

```
i = p;          // error: '=' : cannot convert from 'double *' to 'int'
i = (int)p;     // explicit: double * to int
```




Type Casting Rules: Built-in Type: Numerical Types

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Casting is *safe* for *promotion* (All the data types of the variables are upgraded to the data type of the variable with larger data type)

`bool` → `char` → `short int` → `int` → `unsigned int` → `long` → `unsigned` →
`long long` → `float` → `double` → `long double`

- Casting in built-in types *does not invoke* any conversion function. It only *re-interprets* the binary representation
- Casting is *unsafe* for *demotion* – may lead to loss of data



Type Casting Rules: Built-in Type: Pointer Types

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Implicit casting between different pointer types is not allowed
- Any pointer can be implicitly cast to `void*` (with loss of type); but `void*` cannot be implicitly cast to any pointer type
- Conversion between array and corresponding pointer is not type casting – these are two different syntactic forms for accessing the same data

```
int i = 1, *p = &i, a[10]; double d = 1.1, *q = &d; void *r;
```

```
q = p;           // error: cannot convert 'int*' to 'double*'
p = q;           // error: cannot convert 'double*' to 'int*'
q = (double*)p;  // Okay
p = (int*)q;      // Okay
```

```
r = p;           // Okay to convert from 'int*' to 'void*'
p = r;           // error: invalid conversion from 'void*' to 'int*'
p = (int*)r;      // Okay
```

```
p = a;           // Okay by array pointer duality. p[i], a[i], *(p+i), *(a+i) are equivalent
a = p;           // error: incompatible types in assignment of 'int*' to 'int[10]'
```



Type Casting Rules: Built-in Type: Pointer Types

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Implicit casting between pointer type and numerical type is *not allowed*
- However, explicit casting between pointer and integral type (`int` or `long` etc.) is a common practice to support various tasks like *serialization* (save a file) and *de-serialization* (open a file)
- Care should be taken with these explicit cast to ensure that the integral type is of the *same size* as of the pointer. That is: `sizeof(void*) = sizeof(< integraltype >)`

```
int i, *p = 0; long j;
```

```
// sizeof(i) = sizeof(int) = 4
// sizeof(j) = sizeof(long) = 8
// sizeof(p) = sizeof(int*) = sizeof(void*) = 8
```

```
i = p;          // error: invalid conversion from 'int*' to 'int'
p = i;          // error: invalid conversion from 'int' to 'int*'
```

```
i = (int)p;     // error: cast from 'int*' to 'int' loses precision
p = (int*)i;    // warning: cast to pointer from integer of different size
```

```
j = (long)p;    // Okay
p = (int*)j;    // Okay
```

- Here, the conversion should be done between `int*` and `long` and not between `int*` and `int`



Type Casting Rules: Unrelated Classes

- (**Implicit**) Casting between *unrelated classes is not permitted*

```
class A { int i; };  
class B { double d; };
```

```
A a;  
B b;
```

```
A *p = &a;  
B *q = &b;
```

```
a = b;      // error: binary '=' : no operator which takes a right-hand operand of type 'B'  
a = (A)b;   // error: 'type cast' : cannot convert from 'B' to 'A'
```

```
b = a;      // error: binary '=' : no operator which takes a right-hand operand of type 'A'  
b = (B)a;   // error: 'type cast' : cannot convert from 'A' to 'B'
```

```
p = q;      // error: '=' : cannot convert from 'B *' to 'A *'  
q = p;      // error: '=' : cannot convert from 'A *' to 'B *'
```

```
p = (A*)&b; // explicit on pointer: type cast is okay for the compiler  
q = (B*)&a; // explicit on pointer: type cast is okay for the compiler
```



Type Casting Rules: Unrelated Classes

- **Forced** Casting between *unrelated classes is dangerous*

```
class A { public: int i; };  
class B { public: double d; };
```

```
A a;  
B b;
```

```
a.i = 5;  
b.d = 7.2;
```

```
A *p = &a;  
B *q = &b;
```

```
cout << p->i << endl; // prints 5  
cout << q->d << endl; // prints 7.2
```

```
p = (A*)&b; // Forced casting on pointer: Dangerous  
q = (B*)&a; // Forced casting on pointer: Dangerous
```

```
cout << p->i << endl; // prints -858993459: GARBAGE  
cout << q->d << endl; // prints -9.25596e+061: GARBAGE
```



Type Casting Rules: Inheritance Hierarchy

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Casting on a **hierarchy** is *permitted in a limited sense*

```
class A { };  
class B : public A { };
```

```
A *pa = 0;  
B *pb = 0;  
void *pv = 0;
```

```
pa = pb; // UPCAST: Okay
```

```
pb = pa; // DOWNCAST: error: '=' : cannot convert from 'A *' to 'B *'
```

```
pv = pa; // Okay, but lose the type for A * to void *  
pv = pb; // Okay, but lose the type for B * to void *
```

```
pa = pv; // error: '=' : cannot convert from 'void *' to 'A *'  
pb = pv; // error: '=' : cannot convert from 'void *' to 'B *'
```



Type Casting Rules: Inheritance Hierarchy

- **Up-Casting** is *safe*

```
class A { public: int dataA_; };  
class B : public A { public: int dataB_; };
```

```
A a;  
B b;
```

```
a.dataA_ = 2;  
b.dataA_ = 3;  
b.dataB_ = 5;
```

```
A *pa = &a;  
B *pb = &b;
```

```
cout << pa->dataA_ << endl;           // prints 2  
cout << pb->dataA_ << " " << pb->dataB_ << endl; // prints 3 5
```

```
pa = &b;
```

```
cout << pa->dataA_ << endl;           // prints 3  
cout << pa->dataB_ << endl;           // error: 'dataB_' : is not a member of 'A'
```



Type Casting Rules: Inheritance Hierarchy

- **Down-Casting** is *risky*

```
class A { public: int dataA_; };  
class B : public A { public: int dataB_; };
```

```
A a;  
B b;
```

```
a.dataA_ = 2;  
b.dataA_ = 3;  
b.dataB_ = 5;
```

```
B *pb = (B*)&a;           // Forced downcast
```

```
cout << pb->dataA_ << endl; // prints 2
```

```
cout << pb->dataB_ << endl; // Compilation okay. Prints garbage for 'dataB_' -- no 'dataB_' in 'A' object
```




Module Summary

Module M26

Partha Pratim Das

Weekly Recap

Objectives & Outline

Type Casting

Comparison

Built-in Type

Promotion & Demotion

Unrelated Classes

Inheritance Hierarchy

Upcast

Downcast

Module Summary

- Introduced type casting
- Understood the difference between implicit and explicit type casting
- Introduced the notions of Casting in a class hierarchy – upcast and downcast



Module M27

Partha Pratim
Das

Objectives &
Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

Programming in Modern C++

Module M27: Polymorphism: Part 2: Static and Dynamic Binding

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M27

Partha Pratim
Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- Introduced type casting
- Understood the difference between implicit and explicit type casting
- Introduced the notions of type casting in a class hierarchy – upcast and downcast



Module Objectives

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- Understand Static and Dynamic Binding
- Understand Polymorphic Type

NPTEL



Module Outline

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- 1 Type Binding
 - Type of an Object
 - Static and Dynamic Binding
 - Comparison of Static and Dynamic Binding
 - Static Binding
 - Dynamic Binding
- 2 Polymorphic Type
- 3 Module Summary



Type Binding

Module M27

Partha Pratim
Das

Objectives &
Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

NPTEL

Type Binding



Type of an Object

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- The *static type* of the object is the type declared for the object while writing the code
- Compiler *sees static type*
- The *dynamic type* of the object is determined by the type of the object to which it *refers at run-time*
- Compiler *does not see dynamic type*

```
class A { };  
class B : public A { };  
  
int main() {  
    A *p;  
    p = new B; // Static type of p is A*  
               // Dynamic type of p is B*  
}
```



Static and Dynamic Binding

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

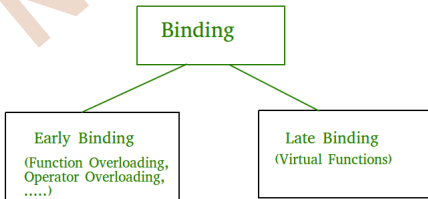
Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- **Static binding (early binding)**: When a function invocation binds to the function definition based on the static type of objects
 - This is done at *compile-time*
 - Normal *function calls*, *overloaded function calls*, and *overloaded operators* are examples of *static binding*
- **Dynamic binding (late binding)**: When a function invocation binds to the function definition based on the dynamic type of objects
 - This is done at *run-time*
 - *Function pointers*, *Virtual functions* are examples of *late binding*





Comparison of Static and Dynamic Binding

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

Basis	Static Binding	Dynamic Binding
● Event Occurrence ● Information ● Advantage ● Time ● Actual Object ● Alternate name ● Example	● Events occur at <i>compile time</i> – <i>Static Binding</i> ● All information needed to call a function is known at <i>compile time</i> ● <i>Efficiency</i> ● <i>Fast execution</i> ● Actual object is <i>not used for binding</i> ● <i>Early Binding</i> ● <i>Method Overloading</i> Normal function call, Overloaded function call, Overloaded operators	● Events occur at <i>run time</i> – <i>Dynamic Binding</i> ● All information needed to call a function is known only at <i>run time</i> ● <i>Flexibility</i> ● <i>Slow execution</i> ● Actual object is <i>used for binding</i> ● <i>Late Binding</i> ● <i>Method Overriding</i> Virtual functions



Static Binding

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

Inherited Method

```
#include<iostream>
using namespace std;
class B { public:
    void f() { }
};
class D : public B { public:
    void g() { } // new function
};
int main() { B b; D d;

    b.f(); // B::f()
    d.f(); // B::f() ----- Inherited
    d.g(); // D::g() ----- Added
}
```

- Object **d** of derived class inherits the base class function **f()** and has its own function **g()**
- Function calls are resolved at compile time based on static type

Overridden Method

```
#include<iostream>
using namespace std;
class B { public:
    void f() { }
};
class D : public B { public:
    void f() { }
};
int main() { B b; D d;

    b.f(); // B::f()
    d.f(); // D::f() ----- Overridden
                        // masks the base class function
}
```

- If a member function of a base class is redefined in a derived class with the same signature then it masks the base class method
- The derived class method **f()** is linked to the object **d**. As **f()** is redefined in the derived class, the base class version cannot be called with the object of a derived class



Member Functions: Overrides and Overloads: RECAP (Module 22)

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

Inheritance

```
class B { public: // Base Class
    void f(int i);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)

    // Inherits B::g(int)
};
B b;
D d;

b.f(1); // Calls B::f(int)
b.g(2); // Calls B::g(int)

d.f(3); // Calls B::f(int)
d.g(4); // Calls B::g(int)
```

- `D::f(int)` overrides `B::f(int)`
- `D::f(string&)` overloads `B::f(int)`

Override & Overload

```
class B { public: // Base Class
    void f(int);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)
    void f(int); // Overrides B::f(int)
    void f(string&); // Overloads B::f(int)
    // Inherits B::g(int)
    void h(int i); // Adds D::h(int)
};
B b;
D d;

b.f(1); // Calls B::f(int)
b.g(2); // Calls B::g(int)

d.f(3); // Calls D::f(int)
d.g(4); // Calls B::g(int)

d.f("red"); // Calls D::f(string&)
d.h(5); // Calls D::h(int)
```



using Construct – Avoid Method Hiding

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

```
#include<iostream>
using namespace std;

class A { public:
    void f() { }
};

class B : public A { public:
    // To overload, rather than hide the base class function f(),
    // it is introduced into the scope of B with a using declaration
    using A::f;
    void f(int) { } // Overloads f()
};

int main() {
    B b; // function calls resolved at compile time

    b.f(3); // B::f(int)
    b.f();  // A::f()
}
```

- Object `b` of derived class linked to with inherited base class function `f()` and the overloaded version defined by the derived class `f(int)`, based on the input parameters – function calls resolved at compile time



Dynamic Binding

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

Non-Virtual Method

```
#include<iostream>
using namespace std;
class B { public:
    void f() { }
};
class D : public B { public:
    void f() { }
};
int main() {
    B b;
    D d;

    B *p;

    p = &b; p->f(); // B::f()
    p = &d; p->f(); // B::f()
}
```

- `p->f()` always binds to `B::f()`
- Binding is decided by the *type of pointer*
- **Static Binding**

Virtual Method

```
#include<iostream>
using namespace std;
class B { public:
    virtual void f() { }
};
class D : public B { public:
    virtual void f() { }
};
int main() {
    B b;
    D d;

    B *p;

    p = &b; p->f(); // B::f()
    p = &d; p->f(); // D::f()
}
```

- `p->f()` binds to `B::f()` for a B object, and to `D::f()` for a D object
- Binding is decided by the *type of object*
- **Dynamic Binding**



Static and Dynamic Binding

Module M27

Partha Pratim
Das

Objectives &
Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

```
#include <iostream>
using namespace std;

class B { public:
    void f() { cout << "B::f()" << endl; }
    virtual void g() { cout << "B::g()" << endl; }
};

class D: public B { public:
    void f() { cout << "D::f()" << endl; }
    virtual void g() { cout << "D::g()" << endl; }
};

int main() { B b; D d;

    B *pb = &b;
    B *pd = &d; // UPGRADE

    B &rb = b;
    B &rd = d; // UPGRADE

    b.f(); // B::f()
    b.g(); // B::g()
    d.f(); // D::f()
    d.g(); // D::g()

    pb->f(); // B::f() -- Static Binding
    pb->g(); // B::g() -- Dynamic Binding
    pd->f(); // B::f() -- Static Binding
    pd->g(); // D::g() -- Dynamic Binding

    rb.f(); // B::f() -- Static Binding
    rb.g(); // B::g() -- Dynamic Binding
    rd.f(); // D::f() -- Static Binding
    rd.g(); // D::g() -- Dynamic Binding

    return 0;
}
```



Polymorphic Type

Module M27

Partha Pratim
Das

Objectives &
Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

NPTEL

Polymorphic Type



Polymorphic Type: Virtual Functions

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- *Dynamic binding* is possible only for pointer and reference data types and for member functions that are declared as **virtual** in the base class
- These are called **Virtual Functions**
- If a member function is declared as virtual, it can be overridden in the derived class
- If a member function is not virtual and it is re-defined in the derived class then the latter definition hides the former one
- Any class containing a virtual member function – by definition or by inheritance – is called a **Polymorphic Type**
- A hierarchy may be *polymorphic* or *non-polymorphic*
- A non-polymorphic hierarchy has little value



Polymorphism Rule

Module M27

Partha Pratim Das

Objectives & Outlines

Type Binding

Type of an Object

Static and Dynamic Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

```
#include <iostream>
using namespace std;
class A { public:
    void f()      { cout << "A::f()" << endl; } // Non-Virtual
    virtual void g() { cout << "A::g()" << endl; } // Virtual
    void h()      { cout << "A::h()" << endl; } // Non-Virtual
};
class B : public A { public:
    void f()      { cout << "B::f()" << endl; } // Non-Virtual
    void g()      { cout << "B::g()" << endl; } // Virtual
    virtual void h() { cout << "B::h()" << endl; } // Virtual
};
class C : public B { public:
    void f()      { cout << "C::f()" << endl; } // Non-Virtual
    void g()      { cout << "C::g()" << endl; } // Virtual
    void h()      { cout << "C::h()" << endl; } // Virtual
};

int main() {
    B *q = new C; A *p = q;

    p->f();
    p->g();
    p->h();
}
```

q->f();	A::f()
q->g();	C::g()
q->h();	A::h()



Module Summary

Module M27

Partha Pratim
Das

Objectives &
Outlines

Type Binding

Type of an Object

Static and Dynamic
Binding

Comparison

Static Binding

Dynamic Binding

Polymorphic Type

Module Summary

- Discussed Static and Dynamic Binding
- Polymorphic type introduced

NPTEL



Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor

Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy

Pure Virtual
Function with Body

Module Summary

Programming in Modern C++

Module M28: Polymorphism: Part 3: Abstract Base Class

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M28

Partha Pratim
Das

Objectives & Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

- Discussed Static and Dynamic Binding
- Polymorphic type introduced

NPTEL



Module Objectives

Module M28

Partha Pratim
Das

Objectives & Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

- Understand why destructor must be **virtual** in a class hierarchy
- Learn to work with class hierarchy

NPTEL



Module Outline

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy
Pure Virtual Function with Body

Module Summary

- 1 Virtual Destructor
 - Slicing
- 2 Pure Virtual Function
- 3 Abstract Base Class
 - Shape Hierarchy
 - Pure Virtual Function with Body
- 4 Module Summary



Virtual Destructor

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor

Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy

Pure Virtual
Function with Body

Module Summary

Virtual Destructor



Virtual Destructor

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor

Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy

Pure Virtual Function with Body

Module Summary

```
#include <iostream>
using namespace std;

class B { int data_; public:
    B(int d) :data_(d) { cout << "B()" << endl; }
    ~B() { cout << "~B()" << endl; }
    virtual void Print() { cout << data_; }
};

class D: public B { int *ptr_; public:
    D(int d1, int d2) :B(d1), ptr_(new int(d2)) { cout << "D()" << endl; }
    ~D() { cout << "~D()" << endl; delete ptr_; }
    void Print() { B::Print(); cout << " " << *ptr_; }
};

int main() {
    B *p = new B(2);
    B *q = new D(3, 5);

    p->Print(); cout << endl;
    q->Print(); cout << endl;

    delete p;
    delete q;
}
```

Output:

```
B()
B()
D()
2
3 5
~B()
~B()
```

Destructor of d (type D) not called!



Virtual Destructor

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor

Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy

Pure Virtual Function with Body

Module Summary

```
#include <iostream>
using namespace std;

class B { int data_; public:
    B(int d) :data_(d) { cout << "B()" << endl; }
    virtual ~B() { cout << "~B()" << endl; } // Destructor made virtual
    virtual void Print() { cout << data_; }
};

class D: public B { int *ptr_; public:
    D(int d1, int d2) :B(d1), ptr_(new int(d2)) { cout << "D()" << endl; }
    ~D() { cout << "~D()" << endl; delete ptr_; }
    void Print() { B::Print(); cout << " " << *ptr_; }
};

int main() {
    B *p = new B(2);
    B *q = new D(3, 5);

    p->Print(); cout << endl;
    q->Print(); cout << endl;

    delete p;
    delete q;
}
```

Output:

```
B()
B()
D()
2
3 5
~B()
~D()
~B()
```

Destructor of d (type D) is called!



Virtual Destructor: Slicing

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor

Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy

Pure Virtual
Function with Body

Module Summary

- **Slicing** is where we assign an object of a derived class to an instance of a base class, thereby losing part of the information - some of it is **sliced** away

```
#include <iostream>
using namespace std;
class Base { protected: int i; public:
    Base(int a)      i = a;
    virtual void display() { cout << "I am Base class object, i = " << i << endl; }
};
class Derived : public Base { int j; public:
    Derived(int a, int b) : Base(a) { j = b; }
    virtual void display() { cout<< "I am Derived class object, i = " << i << ", j = " << j <<endl; }
};
// Global method, Base class object is passed by value
void somefunc (Base obj) { obj.display(); }
int main() { Base b(33); Derived d(45, 54);
    somefunc(b);
    somefunc(d); // Object Slicing, the member j of d is sliced off
}

I am Base class object, i = 33
I am Base class object, i = 45
```

- If the destructor is not **virtual** in a polymorphic hierarchy, it leads to **Slicing**
- **Destructor must be declared virtual in the base class**



Pure Virtual Function

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

Pure Virtual Function



Hierarchy of Shapes

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

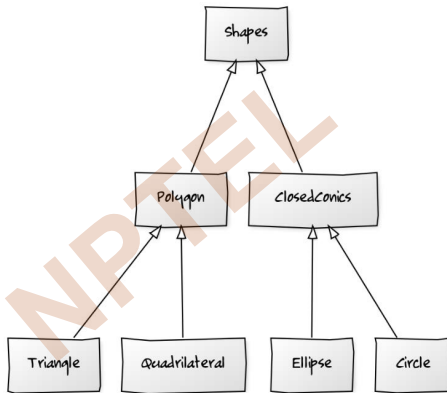
Pure Virtual Function

Abstract Base Class

Shape Hierarchy

Pure Virtual Function with Body

Module Summary



- We want to have a polymorphic `draw()` function for the hierarchy
- `draw()` will be overridden in every class based on the drawing algorithms
- What is the `draw()` function for the root `Shapes` class?



Pure Virtual Function

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy
Pure Virtual Function with Body

Module Summary

- For the polymorphic hierarchy of **Shapes**, we need **draw()** to be a **virtual** function
- **draw()** must be a member of **Shapes** class for polymorphic dispatch to work
- But we cannot define the body of **draw()** function for the root **Shapes** class as we do not have an algorithm to draw an arbitrary shape. In fact, we cannot even have a representation for shapes in general!
- **Pure Virtual Function** solves the problem
- A **Pure Virtual Function** has a signature but no body!
- Example:

```
class Root { public:  
    void f();           // Non-Virtual Function  
    virtual void g();   // Virtual Function  
    virtual void h() = 0; // Pure Virtual Function  
};
```



Abstract Base Class

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

Abstract Base Class



Abstract Base Class

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

- A class containing at least one **Pure Virtual Function** is called an **Abstract Base Class**
- **Pure Virtual Functions** may be inherited or defined in the class
- No instance can be created for an **Abstract Base Class**
- Naturally it may not have a constructor or a **virtual** destructor
- An **Abstract Base Class**, however, may have other **virtual** (non-pure) and non-**virtual** member functions as well as data members
- Data members in an **Abstract Base Class** should be **protected**. Of course, **private** and **public** data are also allowed
- Member functions in an **Abstract Base Class** should be **public**. Of course, **private** and **protected** methods are also allowed
- A **Concrete Class** must override and implement all **Pure Virtual Functions** so that it can be instantiated



Shape Hierarchy

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy
Pure Virtual
Function with Body

Module Summary

```
#include <iostream> // Abstract Base Class shown in red
using namespace std; // Concrete Class shown in green

class Shapes { public:                                // Abstract Base Class
    virtual void draw() = 0; // Pure Virtual Function
};
class Polygon: public Shapes { public: void draw() { cout<< "Polygon: Draw by Triangulation" <<endl; } };
class ClosedConics: public Shapes { public:           // Abstract Base Class
    // draw() inherited - Pure Virtual
};
class Triangle: public Polygon { public: void draw() { cout << "Triangle: Draw by Lines" << endl; } };
class Quadrilateral: public Polygon { public:
    void draw() { cout << "Quadrilateral: Draw by Lines" << endl; }
};
class Circle: public ClosedConics { public:
    void draw() { cout << "Circle: Draw by Bresenham Algorithm" << endl; }
};
class Ellipse: public ClosedConics { public: void draw() { cout << "Ellipse: Draw by ..." << endl; } };
int main() {
    Shapes *arr[] = { new Triangle, new Quadrilateral, new Circle, new Ellipse };

    for (int i = 0; i < sizeof(arr) / sizeof(Shapes *); ++i)
        arr[i]->draw();
    // ...
}
```

Programming in Modern C++



Shape Hierarchy

Module M28

Partha Pratim
Das

Objectives &
Outlines

Virtual
Destructor
Slicing

Pure Virtual
Function

Abstract Base
Class

Shape Hierarchy

Pure Virtual
Function with Body

Module Summary

```
int main() {  
    Shapes *arr[] = { new Triangle, new Quadrilateral, new Circle, new Ellipse };  
  
    for (int i = 0; i < sizeof(arr) / sizeof(Shapes *); ++i)  
        arr[i]->draw();  
    // ...  
    return 0;  
}
```

Triangle: Draw by Lines

Quadrilateral: Draw by Lines

Circle: Draw by Bresenham Algorithm

Ellipse: Draw by ...

- Instances for class **Shapes** and class **ClosedConics** cannot be created



Shape Hierarchy: A Pure Virtual Function may have a body!

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy

Pure Virtual Function with Body

Module Summary

```
#include <iostream>
using namespace std;
class Shapes { public:                                // Abstract Base Class
    virtual void draw() = 0 // Pure Virtual Function
    { cout << "Shapes: Init Brush" << endl; }
};
class Polygon: public Shapes { public:                // Concrete Class
    void draw() { Shapes::draw(); cout << "Polygon: Draw by Triangulation" << endl; }
};
class ClosedConics: public Shapes { public:           // Abstract Base Class
    // draw() inherited - Pure Virtual
};
class Triangle: public Polygon { public:              // Concrete Class
    void draw() { Shapes::draw(); cout << "Triangle: Draw by Lines" << endl; }
};
class Quadrilateral: public Polygon { public:         // Concrete Class
    void draw() { Shapes::draw(); cout << "Quadrilateral: Draw by Lines" << endl; }
};
class Circle: public ClosedConics { public:           // Concrete Class
    void draw() { Shapes::draw(); cout << "Circle: Draw by Bresenham Algorithm" << endl; }
};
class Ellipse: public ClosedConics { public:          // Concrete Class
    void draw() { Shapes::draw(); cout << "Ellipse: Draw by ..." << endl; }
};
```



Shape Hierarchy

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy

Pure Virtual Function with Body

Module Summary

```
int main() {  
    Shapes *arr[] = { new Triangle, new Quadrilateral, new Circle, new Ellipse };  
  
    for (int i = 0; i < sizeof(arr) / sizeof(Shapes *); ++i)  
        arr[i]->draw();  
}
```

Shapes: Init Brush

Triangle: Draw by Lines

Shapes: Init Brush

Quadrilateral: Draw by Lines

Shapes: Init Brush

Circle: Draw by Bresenham Algorithm

Shapes: Init Brush

Ellipse: Draw by ...

- Instances for class **Shapes** and class **ClosedConics** cannot be created
- Some compilers do not allow to inline the function body for a pure **virtual** function

```
class Shapes { public: virtual void draw() = 0 { cout << "Shapes: Init Brush" << endl; } };
```

Outline the function body:

```
class Shapes { public: virtual void draw() = 0; };  
void Shapes::draw() { cout << "Shapes: Init Brush" << endl; }
```



Module Summary

Module M28

Partha Pratim Das

Objectives & Outlines

Virtual Destructor
Slicing

Pure Virtual Function

Abstract Base Class

Shape Hierarchy
Pure Virtual Function with Body

Module Summary

- Discussed why destructors must be **virtual** in a polymorphic hierarchy
- Introduced Pure Virtual Functions
- Introduced Abstract Base Class

NPTEL



Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

Programming in Modern C++

Module M29: Polymorphism: Part 4: Staff Salary Processing using C

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M29

Partha Pratim
Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

- Discussed why destructors must be **virtual** in a polymorphic hierarchy
- Introduced Pure Virtual Functions
- Introduced Abstract Base Class

NPTEL



Module Objectives

Module M29

Partha Pratim
Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

- Understand design with ISA related concepts
- Understand the problems with C design

NPTEL



Module Outline

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

1 Binding: Exercise

- Exercise 1
- Exercise 2

2 Staff Salary Processing

- C Solution
 - Engineer + Manager
 - Engineer + Manager + Director
 - Advantages and Disadvantages

3 Module Summary



Binding: Exercise

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

Binding: Exercise



Binding: Exercise 1

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

// Class Definitions

```
class A { public:
    virtual void f(int) { }
    virtual void g(double) { }
    int h(A *) { }
};

class B: public A { public:
    void f(int) { }
    virtual int h(B *) { }
};

class C: public B { public:
    void g(double) { }
    int h(B *) { }
};
```

// Application Codes

```
A a;
B b;
C c;

A *pA;
B *pB;
```

Invocation	Initialization		
	pA = &a;	pA = &b;	pA = &c;
pA->f(2);			
pA->g(3.2);			
pA->h(&a);			
pA->h(&b);			



Binding: Exercise 1: Solution

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

// Class Definitions

```
class A { public:
    virtual void f(int) { }
    virtual void g(double) { }
    int h(A *) { }
};

class B: public A { public:
    void f(int) { }
    virtual int h(B *) { }
};

class C: public B { public:
    void g(double) { }
    int h(B *) { }
};
```

// Application Codes

```
A a;
B b;
C c;

A *pA;
B *pB;
```

Invocation	Initialization		
	pA = &a;	pA = &b;	pA = &c;
pA->f(2);	A::f	B::f	B::f
pA->g(3.2);	A::g	A::g	C::g
pA->h(&a);	A::h	A::h	A::h
pA->h(&b);	A::h	A::h	A::h



Binding: Exercise 2

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

// Class Definitions

```
class A { public:
    virtual void f(int) { }
    virtual void g(double) { }
    int h(A *) { }
};

class B: public A { public:
    void f(int) { }
    virtual int h(B *) { }
};

class C: public B { public:
    void g(double) { }
    int h(B *) { }
};
```

// Application Codes

```
A a;
B b;
C c;

A *pA;
B *pB;
```

Invocation	Initialization		
	pB = &a;	pB = &b;	pB = &c;
pB->f(2);			
pB->g(3.2);			
pB->h(&a);			
pB->h(&b);			



Binding: Exercise 2: Solution

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

// Class Definitions

```
class A { public:
    virtual void f(int) { }
    virtual void g(double) { }
    int h(A *) { }
};

class B: public A { public:
    void f(int) { }
    virtual int h(B *) { }
};

class C: public B { public:
    void g(double) { }
    int h(B *) { }
};
```

// Application Codes

```
A a;
B b;
C c;

A *pA;
B *pB;
```

Invocation	Initialization		
	pB = &a;	pB = &b;	pB = &c;
pB->f(2);	Error	B::f	B::f
pB->g(3.2);	Downcast	A::g	C::g
pB->h(&a);	(A *) to	No conversion (A *) to (B *)	
pB->h(&b);	(B *)	B::h	C::h



Staff Salary Processing

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

Staff Salary Processing



Staff Salary Processing: Problem Statement

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

- An organization needs to develop a salary processing application for its staff
- At present it has an engineering division only where **Engineers** and **Managers** work. Every **Engineer** reports to some **Manager**. Every **Manager** can also work like an **Engineer**
- The logic for processing salary for **Engineers** and **Managers** are different as they have different salary heads
- In future, it may add **Directors** to the team. Then every **Manager** will report to some **Director**. Every **Director** could also work like a **Manager**
- The logic for processing salary for **Directors** will also be distinct
- Further, in future it may open other divisions, like Sales division, and expand the workforce
- **Make a suitable extensible design**



C Solution: Function Switch: Engineer + Manager

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

- How to represent **Engineers** and **Managers**?
 - Collection of **structs**
- How to initialize objects?
 - Initialization functions
- How to have a collection of mixed objects?
 - Array of **union**
- How to model variations in salary processing algorithms?
 - **struct**-specific functions
- How to invoke the correct algorithm for a correct employee type?
 - Function Switch
 - Function Pointers



C Solution: Function Switch: Engineer + Manager

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef enum E_TYPE { Er, Mgr } E_TYPE; // Tag for type of staff

typedef struct Engineer { char *name_; } Engineer;
Engineer *InitEngineer(const char *name) {
    Engineer *e = (Engineer *)malloc(sizeof(Engineer));
    e->name_ = strdup(name); return e;
}
void ProcessSalaryEngineer(Engineer *e) { printf("%s: Process Salary for Engineer\n", e->name_); }

typedef struct Manager { char *name_; Engineer *reports_[10]; } Manager;
Manager *InitManager(const char *name) {
    Manager *m = (Manager *)malloc(sizeof(Manager));
    m->name_ = strdup(name); return m;
}
void ProcessSalaryManager(Manager *m) { printf("%s: Process Salary for Manager\n", m->name_); }

typedef struct Staff { // Aggregation of staffs
    E_TYPE type_;
    union { Engineer *pE; Manager *pM; };
} Staff;
```



C Solution: Function Switch: Engineer + Manager

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

```
int main() {
    Staff allStaff[10];
    allStaff[0].type_ = Er;  allStaff[0].pE = InitEngineer("Rohit");
    allStaff[1].type_ = Mgr; allStaff[1].pM = InitManager("Kamala");
    allStaff[2].type_ = Mgr; allStaff[2].pM = InitManager("Rajib");
    allStaff[3].type_ = Er;  allStaff[3].pE = InitEngineer("Kavita");
    allStaff[4].type_ = Er;  allStaff[4].pE = InitEngineer("Shambhu");

    for (int i = 0; i < 5; ++i) {
        E_TYPE t = allStaff[i].type_;
        if (t == Er)
            ProcessSalaryEngineer(allStaff[i].pE);
        else if (t == Mgr)
            ProcessSalaryManager(allStaff[i].pM);
        else printf("Invalid Staff Type\n");
    }
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer



C Solution: Function Switch: Engineer + Manager + Director

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

- How to represent **Engineers**, **Managers**, and **Directors**?
 - Collection of **structs**
- How to initialize objects?
 - Initialization functions
- How to have a collection of mixed objects?
 - Array of **union**
- How to model variations in salary processing algorithms?
 - **struct**-specific functions
- How to invoke the correct algorithm for a correct employee type?
 - Function switch
 - Function pointers



C Solution: Function Switch: Engineer + Manager + Director

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef enum E_TYPE { Er, Mgr, Dir } E_TYPE;

typedef struct Engineer { char *name_; } Engineer;
Engineer *InitEngineer(const char *name) { Engineer *e = (Engineer *)malloc(sizeof(Engineer));
    e->name_ = strdup(name); return e;
}
void ProcessSalaryEngineer(Engineer *e) { printf("%s: Process Salary for Engineer\n", e->name_); }

typedef struct Manager { char *name_; Engineer *reports_[10]; } Manager;
Manager *InitManager(const char *name) { Manager *m = (Manager *)malloc(sizeof(Manager));
    m->name_ = strdup(name); return m;
}
void ProcessSalaryManager(Manager *m) { printf("%s: Process Salary for Manager\n", m->name_); }

typedef struct Director { char *name_; Manager *reports_[10]; } Director;
Director *InitDirector(const char *name) { Director *d = (Director *)malloc(sizeof(Director));
    d->name_ = strdup(name); return d;
}
void ProcessSalaryDirector(Director *d) { printf("%s: Process Salary for Director\n", d->name_); }

typedef struct Staff { E_TYPE type_; union { Engineer *pE; Manager *pM; Director *pD; };
} Staff;
Programming in Modern C++
```



C Solution: Function Switch: Engineer + Manager + Director

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

```
int main() { Staff allStaff[10];
    allStaff[0].type_ = Er; allStaff[0].pE = InitEngineer("Rohit");
    allStaff[1].type_ = Mgr; allStaff[1].pM = InitManager("Kamala");
    allStaff[2].type_ = Mgr; allStaff[2].pM = InitManager("Rajib");
    allStaff[3].type_ = Er; allStaff[3].pE = InitEngineer("Kavita");
    allStaff[4].type_ = Er; allStaff[4].pE = InitEngineer("Shambhu");
    allStaff[5].type_ = Dir; allStaff[5].pD = InitDirector("Ranjana");

    for (int i = 0; i < 6; ++i) { E_TYPE t = allStaff[i].type_;
        if (t == Er)
            ProcessSalaryEngineer(allStaff[i].pE);
        else if (t == Mgr)
            ProcessSalaryManager(allStaff[i].pM);
        else if (t == Dir)
            ProcessSalaryDirector(allStaff[i].pD);
        else printf("Invalid Staff Type\n");
    }
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director

Programming in Modern C++

Partha Pratim Das

M29.17



C Solution: Function Switch: Engineer + Manager + Director

Instead of if-else chain, we can use switch to explicitly switch on the type of employee

```
int main() { Staff allStaff[10];
    allStaff[0].type_ = Er; allStaff[0].pE = InitEngineer("Rohit");
    allStaff[1].type_ = Mgr; allStaff[1].pM = InitManager("Kamala");
    allStaff[2].type_ = Mgr; allStaff[2].pM = InitManager("Rajib");
    allStaff[3].type_ = Er; allStaff[3].pE = InitEngineer("Kavita");
    allStaff[4].type_ = Er; allStaff[4].pE = InitEngineer("Shambhu");
    allStaff[5].type_ = Dir; allStaff[5].pD = InitDirector("Ranjana");

    for (int i = 0; i < 6; ++i) { E_TYPE t = allStaff[i].type_;
        switch (t) {
            case Er: ProcessSalaryEngineer(allStaff[i].pE); break;
            case Mgr: ProcessSalaryManager(allStaff[i].pM); break;
            case Dir: ProcessSalaryDirector(allStaff[i].pD); break;
            default: printf("Invalid Staff Type\n"); break;
        }
    }
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director

Programming in Modern C++

Partha Pratim Das

M29.18

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary



C Solution: Advantages and Disadvantages

Module M29

Partha Pratim Das

Objectives & Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary Processing

C Solution

Engineer + Manager

Engineer + Manager + Director

Advantages and Disadvantages

Module Summary

- **Advantages**

- Solution exists!
- Code is well structured – has patterns

- **Disadvantages**

- Employee data has scope for better organization
 - ▷ No encapsulation for data
 - ▷ Duplication of fields across types of employees – possible to mix up types for them (say, `char *` and `string`)
 - ▷ Employee objects are created and initialized dynamically through `Init...` functions. How to release the memory?
- Types of objects are managed explicitly by `E_Type`:
 - ▷ Difficult to extend the design – addition of a new type needs to:
 - Add new type code to `enum E_Type`
 - Add a new pointer field in `struct Staff` for the new type
 - Add a new case (`if-else` or `case`) based on the new type
 - ▷ Error prone – developer has to decide to call the right processing function for every type (`ProcessSalaryManager` for `Mgr` etc.)
- Unable to use Function Pointers as each processing function takes a parameter of different type - no common signature for dispatch

- **Recommendation**

- Use classes for encapsulation on a hierarchy



Module Summary

Module M29

Partha Pratim
Das

Objectives &
Outlines

Binding: Exercise

Exercise 1

Exercise 2

Staff Salary
Processing

C Solution

Engineer +
Manager

Engineer +
Manager + Director

Advantages and
Disadvantages

Module Summary

- Practiced exercise with binding – various mixed cases
- Started designing for a staff salary problem and worked out C solutions



Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

Programming in Modern C++

Module M30: Polymorphism: Part 5: Staff Salary Processing using C++

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M30

Partha Pratim
Das

Objectives & Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

- Practiced exercise with binding – various mixed cases
- Started designing for a staff salary problem and worked out C solutions

NPTEL



Module Objectives

Module M30

Partha Pratim
Das

Objectives & Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

- Understand design with class hierarchy
- Understand the process of design refinement to get to a good solution from a starting one



Module Outline

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

1 Staff Salary Processing: C Solution

- Flat C Solution: Recap
 - Advantages and Disadvantages

2 Staff Salary Processing: C++ Solution

- Non-Polymorphic Hierarchy
 - Advantages and Disadvantages
- Polymorphic Hierarchy
 - Advantages and Disadvantages
- Polymorphic Hierarchy (Flexible)
 - Advantages and Disadvantages

3 Module Summary



Staff Salary Processing: C Solution

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

Staff Salary Processing: C Solution



Staff Salary Processing: Problem Statement: RECAP (Module 29)

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

- An organization needs to develop a salary processing application for its staff
- At present it has an engineering division only where **Engineers** and **Managers** work. Every **Engineer** reports to some **Manager**. Every **Manager** can also work like an **Engineer**
- The logic for processing salary for **Engineers** and **Managers** are different as they have different salary heads
- In future, it may add **Directors** to the team. Then every **Manager** will report to some **Director**. Every **Director** could also work like a **Manager**
- The logic for processing salary for **Directors** will also be distinct
- Further, in future it may open other divisions, like Sales division, and expand the workforce
- **Make a suitable extensible design**



C Solution: Engineer + Manager + Director: RECAP (Module 29)

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

- How to represent **Engineers**, **Managers**, and **Directors**?
 - Collection of **structs**
- How to initialize objects?
 - Initialization functions
- How to have a collection of mixed objects?
 - Array of **union**
- How to model variations in salary processing algorithms?
 - **struct**-specific functions
- How to invoke the correct algorithm for a correct employee type?
 - Function switch
 - Function pointers



C Solution: Advantages and Disadvantages: RECAP (Module 29)

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

- **Advantages**

- Solution exists!
- Code is well structured – has patterns

- **Disadvantages**

- Employee data has scope for better organization
 - ▷ No encapsulation for data
 - ▷ Duplication of fields across types of employees – possible to mix up types for them (say, `char *` and `string`)
 - ▷ Employee objects are created and initialized dynamically through `Init...` functions. How to release the memory?
- Types of objects are managed explicitly by `E.Type`:
 - ▷ Difficult to extend the design – addition of a new type needs to:
 - Add new type code to `enum E.Type`
 - Add a new pointer field in `struct Staff` for the new type
 - Add a new case (`if-else` or `case`) based on the new type
 - ▷ Error prone – developer has to decide to call the right processing function for every type (`ProcessSalaryManager` for `Mgr` etc.)
- Unable to use Function Pointers as each processing function takes a parameter of different type - no common signature for dispatch

- **Recommendation**

- Use classes for encapsulation on a hierarchy



Staff Salary Processing: C++ Solution

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

Staff Salary Processing: C++ Solution



C++ Solution: Non-Polymorphic Hierarchy: Engineer + Manager

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary



- How to represent **Engineers** and **Managers**?
 - Non-Polymorphic class hierarchy
- How to initialize objects?
 - Constructor / Destructor
- How to have a collection of mixed objects?
 - array of base class pointers
- How to model variations in salary processing algorithms?
 - Member functions
- How to invoke the correct algorithm for a correct employee type?
 - Function switch
 - Function pointers



C++ Solution: Non-Polymorphic Hierarchy: Engineer + Manager

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

```
#include <iostream>
#include <string>
using namespace std;

enum E_TYPE { Er, Mgr };

class Engineer {
protected:
    string name_; E_TYPE type_;
public:
    Engineer(const string& name, E_TYPE e = Er) : name_(name), type_(e) { }
    E_TYPE GetType() { return type_; }
    void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};

class Manager : public Engineer {
    Engineer *reports_[10];
public:
    Manager(const string& name, E_TYPE e = Mgr) : Engineer(name, e) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};
```



C++ Solution: Non-Polymorphic Hierarchy Engineer + Manager

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

```
int main() {
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");
    Manager m1("Kamala"), m2("Rajib");
    Engineer *staff[] = { &e1, &m1, &m2, &e2, &e3 };

    for (int i = 0; i < sizeof(staff) / sizeof(Engineer*); ++i) {
        E_TYPE t = staff[i]->GetType();
        if (t == Er)
            staff[i]->ProcessSalary();
        else if (t == Mgr)
            ((Manager *)staff[i])->ProcessSalary();
        else cout << "Invalid Staff Type" << endl;
    }
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer



C++ Solution: Non-Polymorphic Hierarchy: Engineer + Manager + Director

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary



- How to represent **Engineers**, **Managers**, and **Directors**?
 - Non-Polymorphic class hierarchy
- How to initialize objects?
 - Constructor / Destructor
- How to have a collection of mixed objects?
 - array of base class pointers
- How to model variations in salary processing algorithms?
 - Member functions
- How to invoke the correct algorithm for a correct employee type?
 - Function switch
 - Function pointers



C++ Solution: Non-Polymorphic Hierarchy Engineer + Manager + Director

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

```
#include <iostream>
#include <string>
using namespace std;
enum E_TYPE { Er, Mgr, Dir };

class Engineer {
protected:
    string name_; E_TYPE type_;
public:
    Engineer(const string& name, E_TYPE e = Er) : name_(name), type_(e) {}
    E_TYPE GetType() { return type_; }
    void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};

class Manager : public Engineer {
    Engineer *reports_[10];
public:
    Manager(const string& name, E_TYPE e = Mgr) : Engineer(name, e) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};

class Director : public Manager {
    Manager *reports_[10];
public:
    Director(const string& name) : Manager(name, Dir) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};
```

Programming in Modern C++



C++ Solution: Non-Polymorphic Hierarchy Engineer + Manager + Director

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

```
int main() {  
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");  
    Manager m1("Kamala"), m2("Rajib");  
    Director d("Ranjana");  
    Engineer *staff[] = { &e1, &m1, &m2, &e2, &e3, &d };  
  
    for (int i = 0; i < sizeof(staff) / sizeof(Engineer*); ++i) {  
        E_TYPE t = staff[i]->GetType();  
        if (t == Er)  
            staff[i]->ProcessSalary();  
        else if (t == Mgr)  
            ((Manager *)staff[i])->ProcessSalary();  
        else if (t == Dir)  
            ((Director *)staff[i])->ProcessSalary();  
        else cout << "Invalid Staff Type" << endl;  
    }  
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director



C++ Solution: Non-Polymorphic Hierarchy: Advantages and Disadvantages

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

- **Advantages**

- Data is encapsulated
- Hierarchy factors common data members
- Constructor / Destructor to manage lifetime
- **struct**-specific functions made member function (overridden)
- **E_Type** subsumed in **class** – no need for **union**
- Code reuse evidenced

- **Disadvantages**

- Types of objects are managed explicitly by **E_Type**:
 - ▷ Difficult to extend the design – addition of a new type needs to:
 - Add new type code to **enum E_Type**
 - Application code need to have a new case (**if-else**) based on the new type
 - ▷ Error prone because the application programmer has to cast to right type to call **ProcessSalary**

- **Recommendation**

- Use a polymorphic hierarchy with dynamic dispatch



C++ Solution: Polymorphic Hierarchy

Engineer + Manager + Director

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

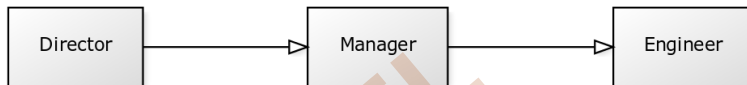
Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary



- How to represent **Engineers**, **Managers**, and **Directors**?
 - Polymorphic class hierarchy
- How to initialize objects?
 - Constructor / Destructor
- How to have a collection of mixed objects?
 - array of base class pointers
- How to model variations in salary processing algorithms?
 - Member functions
- How to invoke the correct algorithm for a correct employee type?
 - Virtual Functions



C++ Solution: Polymorphic Hierarchy Engineer + Manager + Director

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

```
#include <iostream>
#include <string>
using namespace std;

class Engineer {
protected:
    string name_;
public:
    Engineer(const string& name) : name_(name) {}
    virtual void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};

class Manager : public Engineer {
    Engineer *reports_[10];
public:
    Manager(const string& name) : Engineer(name) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};

class Director : public Manager {
    Manager *reports_[10];
public:
    Director(const string& name) : Manager(name) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};

Programming in Modern C++
```



C++ Solution: Polymorphic Hierarchy

Engineer + Manager + Director

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

```
int main() {
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");
    Manager m1("Kamala"), m2("Rajib");
    Director d("Ranjana");
    Engineer *staff[] = { &e1, &m1, &m2, &e2, &e3, &d };

    for (int i = 0; i < sizeof(staff) / sizeof(Engineer*); ++i)
        staff[i]->ProcessSalary();
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director



C++ Solution: Polymorphic Hierarchy: Advantages and Disadvantages

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

- **Advantages**

- Data is fully encapsulated
- Polymorphic Hierarchy removes the need for explicit `E_Type`
- Application code is independent of types in the system (`virtual` functions manage types through polymorphic dispatch)
- High Code reuse – code is short and simple

- **Disadvantages**

- Difficult to add an employee type that is not a part of this hierarchy (for example, employees of *Sales Division*)

- **Recommendation**

- Use an abstract base class for employees



C++ Solution: Polymorphic Hierarchy (Flexible)

Engineer + Manager + Director + Others

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

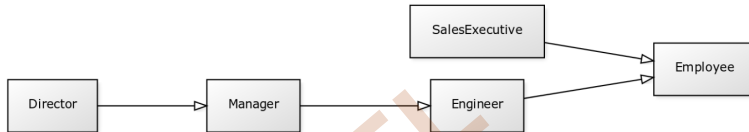
Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary



- How to represent **Engineers**, **Managers**, **Directors**, etc.?
 - Polymorphic class hierarchy with an Abstract Base **Employee**
- How to initialize objects?
 - Constructor / Destructor
- How to have a collection of mixed objects?
 - array of base class pointers
- How to model variations in salary processing algorithms?
 - Member functions
- How to invoke the correct algorithm for a correct employee type?
 - Virtual Functions (Pure in **Employee**)



C++ Solution: Polymorphic Hierarchy (Flexible)

Engineer + Manager + Director + Others

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

```
#include <iostream>
#include <string>
using namespace std;
class Employee {
protected: string name_;
public:
    virtual void ProcessSalary() = 0;
    virtual ~Employee() { }
};
class Engineer: public Employee { public:
    Engineer(const string& name) { name_ = name; }
    void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};
class Manager : public Engineer { Engineer *reports_[10]; public:
    Manager(const string& name) : Engineer(name) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};
class Director : public Manager { Manager *reports_[10]; public:
    Director(const string& name) : Manager(name) {}
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};
class SalesExecutive : public Employee { public:
    SalesExecutive(const string& name) { name_ = name; }
    void ProcessSalary() { cout << name_ << ": Process Salary for Sales Executive" << endl; }
};
```

Programming in Modern C++



C++ Solution: Polymorphic Hierarchy (Flexible)

Engineer + Manager + Director + Others

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

```
int main() {
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");
    Manager m1("Kamala"), m2("Rajib");
    SalesExecutive s1("Hari"), s2("Bishnu");
    Director d("Ranjana");

    Employee *staff[] = { &e1, &m1, &m2, &e2, &s1, &e3, &d, &s2 };

    for (int i = 0; i < sizeof(staff) / sizeof(Employee*); ++i)
        staff[i]->ProcessSalary();
}
```

Rohit: Process Salary for Engineer
Kamala: Process Salary for Manager
Rajib: Process Salary for Manager
Kavita: Process Salary for Engineer
Hari: Process Salary for Sales Executive
Shambhu: Process Salary for Engineer
Ranjana: Process Salary for Director
Bishnu: Process Salary for Sales Executive



C++ Solution: Polymorphic Hierarchy (Flexible): Advantages and Disadvantages

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

- **Advantages**

- Data is fully encapsulated
- Flexible Polymorphic Hierarchy makes addition of any class possible on the hierarchy
- Application code is independent of types in the system (**virtual** functions manage types through polymorphic dispatch)
- Maximum Code reuse – code is short and simple

- **Disadvantages**

- Still needs to maintain employee objects in code and add them to the staff array - this is error prone

- **Recommendation**

- Use vector as a collection and insert staff as created



C++ Solution: Polymorphic Hierarchy (Flexible)

Engineer + Manager + Director + Others

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
class Employee { protected: string name_; // Name of the employee
                 vector<Employee*> reports_; // Collection of reportees aggregated
public: virtual void ProcessSalary() = 0; // Processing salary
       virtual ~Employee() { }
       static vector<Employee*> staffs; // Collection of all staffs
       void AddStaff(Employee* e) { staffs.push_back(e); }; // Add a staff to collection
};
class Engineer : public Employee { public:
    Engineer(const string& name) { name_ = name; // Why init like name_(name) won't work?
                                   AddStaff(this); } // Add the staff
    void ProcessSalary() { cout << name_ << ": Process Salary for Engineer" << endl; }
};
class Manager : public Engineer { public: Manager(const string& name) : Engineer(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Manager" << endl; }
};
class Director : public Manager { public: Director(const string& name) : Manager(name) { }
    void ProcessSalary() { cout << name_ << ": Process Salary for Director" << endl; }
};
class SalesExecutive : public Employee { public:
    SalesExecutive(const string& name) { name_ = name; AddStaff(this); } // Add the staff
    void ProcessSalary() { cout << name_ << ": Process Salary for Sales Executive" << endl; }
};
```



C++ Solution: Polymorphic Hierarchy (Flexible)

Engineer + Manager + Director + Others

Module M30

Partha Pratim Das

Objectives & Outlines

Staff Salary Processing: C Solution

Flat C Solution: Recap

Advantages and Disadvantages

Staff Salary Processing: C++ Solution

Non-Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy

Advantages and Disadvantages

Polymorphic Hierarchy (Flexible)

Advantages and Disadvantages

Module Summary

```
vector<Employee*> Employee::staffs;           // Collection of all staffs

int main() {
    Engineer e1("Rohit"), e2("Kavita"), e3("Shambhu");
    Manager m1("Kamala"), m2("Rajib");
    SalesExecutive s1("Hari"), s2("Bishnu");
    Director d("Ranjana");

    vector<Employee*>::const_iterator it;      // Iterator over staffs

    for (it = Employee::staffs.begin();       // Iterate on staffs
         it < Employee::staffs.end();
         ++it)
        (*it)->ProcessSalary();              // Process respective salary
}
```

Rohit: Process Salary for Engineer

Kavita: Process Salary for Engineer

Shambhu: Process Salary for Engineer

Kamala: Process Salary for Manager

Rajib: Process Salary for Manager

Hari: Process Salary for Sales Executive

Bishnu: Process Salary for Sales Executive

Ranjana: Process Salary for Director



C++ Solution: Polymorphic Hierarchy (Flexible): Advantages and Disadvantages

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

- **Advantages**

- Data is fully encapsulated
- Flexible Polymorphic Hierarchy makes addition of any class possible on the hierarchy
- Application code is independent of types in the system (**virtual** functions manage types through polymorphic dispatch)
- Maximum Code reuse – code is short and simple
- Collection of staff encapsulated with creation
- **vector** and **iterator** increases efficiency and efficacy

- **Disadvantages**

- None in particular

- **Recommendation**

- Enjoy the solution



Module Summary

Module M30

Partha Pratim
Das

Objectives &
Outlines

Staff Salary
Processing: C
Solution

Flat C Solution:
Recap

Advantages and
Disadvantages

Staff Salary
Processing: C++
Solution

Non-Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy

Advantages and
Disadvantages

Polymorphic
Hierarchy (Flexible)

Advantages and
Disadvantages

Module Summary

- Completed design for a staff salary problem using hierarchy and worked out extensible C++ solution
- Learnt about iterative refinement of solutions in the process



Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

Programming in Modern C++

Tutorial T06: Mixing C and C++ Code: Part 2: Project Example

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Recap

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- We have learnt why is it often necessary to mix C and C++ codes in the same project
- We have explored the basic issues of mixing and learnt the ground rules
- In addition to the rules, we have three mechanisms to ease code mixing
 - Use `extern "C"` in C++ for all functions to be called from both C and C++
 - Guard `extern "C"` with `__cplusplus` guard for use with C
 - Provide `wrappers` for C++ data members, member functions, and overloaded functions for use with C



Tutorial Objectives

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- Walk through a C / C++ mix project using the rules and scenarios of mixing

NPTEL



Tutorial Outline

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- 1 Tutorial Recap
- 2 Mixing C and C++ Codes
 - Rules
 - Common Code Mix Scenarios
- 3 How do I manipulate with objects in a C / C++ mix project?
 - Data.h
 - Data.cpp
 - Data_Wrap.cpp
 - App.c
 - main.cpp
 - makefile
 - Execution
 - Call Trace
- 4 Advanced Code Mix Scenarios and Advisory
- 5 Tutorial Summary



Mixing C and C++ Codes

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

Mixing C and C++ Codes

Source: Accessed 16-Sep-21

[How to mix C and C++, ISO CPP](#)

[Mixing C and C++ Code in the Same Program, Oracle](#)

[C++ Core Guidelines: Mixing C with C++](#)

[Mixing Code in C, C++, and FORTRAN on Unix](#)

Programming in Modern C++

Partha Pratim Das

T06.5



Mixing C and C++ Codes: Rules: Recap

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- **RULE 1:** Use **C++ compiler** when **compiling** `main()`
- **RULE 2:** **C and C++ compilers must be compatible**
- **RULE 3:** **C++ compiler should direct the linking process**



Mixing C and C++ Codes: Scenarios: Recap

Tutorial T06

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- How do I call a C function from C++?

```
extern "C" { void f(int); };
```

- How do I call a C++ function from C?

- Non-Member

```
extern "C" { void f(int); }
```

- Member

```
class C { /*...*/  
    virtual double f(int);  
};  
// wrapper function  
extern "C" double call_C_f(C* p, int i)  
{ return p->f(i); }
```

- Overloaded

```
void f(int);  
void f(double);  
// wrapper functions  
extern "C" {  
    void f_i(int i) { f(i); }  
    void f_d(double d) { f(d); }  
}
```

- How do I include a C Header File?

- System / Standard Library Headers
- Non-System Headers: Editable

```
#ifdef __cplusplus /* C compilers skip */  
extern "C" {  
#endif  
/* Original Code of the Header */  
#ifdef __cplusplus  
}  
#endif
```

- Non-System Headers: Non-Editable

```
// In C++ header / source  
extern "C" {  
    #include "my-C-code.h" // C Header  
}
```

- How do I use Pointers to C / C++ Functions?

```
extern "C" {  
    typedef int (*pfun)(int);  
    void foo(pfun);  
    int g(int); // foo(g) is valid  
}
```



C / C++ Mixed Project

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

C / C++ Mixed Project



C / C++ Mixed Project: Manipulating with Objects from C

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data.Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- We present an example project comprising the following files to summarize various code mixing scenarios in an integrated manner:
 - [Data.h](#): C/C++ common header containing:
 - ▷ definition of `class Data`
 - ▷ prototypes of C functions to interact with `class Data`, and
 - ▷ prototypes of C++ wrappers providing access points for C to call member functions in `class Data`
 - [Data.cpp](#): Implementations of `class Data`
 - [Data.Wrap.cpp](#): Implementations of C++ wrapper functions for `class Data`
 - [App.c](#): Implementations of C functions for interacting with `class Data`
 - [main.cpp](#): `main` to invoke the C functions
 - [makefile](#): Mix build of C and C++, and C++ link script



C / C++ Mix Project: Mix Scenarios

Tutorial T06

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- Calling C functions from C++ ([main.cpp](#))
 - `Data* c_create_object(int);` /* C function to create an object */
 - `void c_access_object(Data*);` /* C function to access an object */
 - `void c_release_object(Data*);` /* C function to release an object */
- Calling C++ functions from C ([App.c](#))
 - `Data* call_create(int);` /* C++ wrapper to create an object by new */
 - `int call_get(Data*);` /* C++ wrapper to get the state of an object by get */
 - `void call_set(Data*, int);` /* C++ wrapper to change the state of an object by set */
 - `void call_release(Data*);` /* C++ wrapper to release an object by delete */
- Passing an object from C to C++ ([main.cpp](#))
 - `Data* c_create_object(int);` /* C function to create an object */
- Passing an object from C++ to C ([main.cpp](#))
 - `void c_access_object(Data*);` /* C function to access an object */
 - `void c_release_object(Data*);` /* C function to release an object */
- C++ wrappers for object creations, get / set, and release ([Data_Wrap.cpp](#))
- C functions for object creations, get / set, and release ([App.c](#))
- Common header for C and C++ ([Data.h](#))
 - `typedef struct Data Data;` /* Incomplete Type to access Data* in C function */



C / C++ Mix Project: Data.h

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```

/* C Header and C++ Header Data.h - can be read by both C and C++ compilers */
#ifndef __DATA_H    /* include Guard */
#define __DATA_H
#ifdef __cplusplus /* Guard for C++ */
    class Data { int d_;                // Private data member
    public: Data(int=0); ~Data();        // Public members: Constructor and Destructor
           int get(); void set(int);    // get and set members
    };
#else /* Guard for C */
    typedef struct Data Data;           // Incomplete Type to access Data* in C function */
#endif
#ifdef __cplusplus /* Guard for C++ */
extern "C" { /* Linkage for C */
#endif
    extern Data* c_create_object(int); /* C function to create an object */
    extern void c_access_object(Data*); /* C function to access an object */
    extern void c_release_object(Data*); /* C function to release an object */
    extern Data* call_create(int); /* C++ wrapper to create an object by new */
    extern int call_get(Data* data); /* C++ wrapper to get state of an object by get */
    extern void call_set(Data* data, int d); /* C++ wrapper to change state of an object by set */
    extern void call_release(Data*); /* C++ wrapper to release an object by delete */
#ifdef __cplusplus /* Guard for C++ */
}
#endif
#endif /* __DATA_H */
Programming in Modern C++

```



C / C++ Mix Project: Data.cpp

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
// C++ code: Data.cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
#include "Data.h"
```

```
// Class Data implementation
```

```
Data::Data(int d): d_(d)
```

```
{ cout << "Created " << d_ << endl; }
```

```
Data::~Data()
```

```
{ cout << "Released " << d_ << endl; }
```

```
int Data::get()
```

```
{ return d_; }
```

```
void Data::set(int d)
```

```
{ d_ = d; }
```




C / C++ Mix Project: Data_Wrap.cpp

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
// C++ code: Data_Wrap.cpp
#include "Data.h"
```

```
/* C++ wrapper to create an object by new */
Data* call_create(int d)
{ return new Data(d); }
```

```
/* C++ wrapper to get state of an object by get */
int call_get(Data* data)
{ return data->get(); }
```

```
/* C++ wrapper to change state of an object by set */
void call_set(Data* data, int d)
{ return data->set(d); }
```

```
/* C++ wrapper to release an object by delete */
void call_release(Data* data)
{ delete data; }
```



C / C++ Mix Project: App.c

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
/* C code */: App.c
```

```
#include <stdio.h>
```

```
#include "Data.h"
```

```
Data* c_create_object(int d) {                               /* C function to create an object */  
    return call_create(d);  
}
```

```
void c_access_object(Data* data) {                             /* C function to access an object */  
    printf("Get data %d\n", call_get(data));  
    call_set(data, 7);  
    printf("Set data %d\n", call_get(data));  
}
```

```
void c_release_object(Data* data) {                           /* C function to release an object */  
    call_release(data);  
}
```



C / C++ Mix Project: main.cpp

Tutorial T06

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
// C++ code: main.cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
#include "Data.h"
```

```
Data d(10);
```

```
int main() {
```

```
    Data* p = c_create_object(5);
```

```
    c_access_object(p);
```

```
    c_release_object(p);
```

```
}
```

```
/* C function to create an object */
```

```
/* C function to access an object */
```

```
/* C function to release an object */
```



C / C++ Mix Project: makefile

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
# Compiles .c by C and .cpp by C++. Links by C
CC=gcc
```

```
# Compiles .c and .cpp by C++. Links by C++
CPP=g++
CFLAGS=-I.
DEPS = Data.h
```

```
# Build .c by gcc (C Rules)
%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
```

```
# Build .cpp by gcc (C++ Rules). May use $(CPP)$ for g++ also
%.o: %.cpp $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)
```

```
# Link by g++ (C++ Linkage)
Data: main.o Data.o App.o Data_Wrap.o
    $(CPP) -o Data main.o Data.o App.o Data_Wrap.o
```

```
.PHONY: clean
```

```
clean:
    del *.o *.exe
```



C / C++ Mix Project: Execution

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
// Build by make
```

```
$ make
```

```
gcc -c -o main.o main.cpp -I.
```

```
gcc -c -o Data.o Data.cpp -I.
```

```
gcc -c -o App.o App.c -I.
```

```
gcc -c -o Data_Wrap.o Data_Wrap.cpp -I.
```

```
g++ -o Data main.o Data.o App.o Data_Wrap.o
```

```
// C++ Compile
```

```
// C++ Compile
```

```
// C Compile
```

```
// C++ Compile
```

```
// C++ Link
```

```
// Execute
```

```
$ Data.exe
```

```
Created 10
```

```
Created 5
```

```
Get data 5
```

```
Set data 7
```

```
Released 7
```

```
Released 10
```



C / C++ Mix Project: Call Trace

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

DataWrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

```
// Trace
START()
    Data::Data(int)
        // Special start-up function to initialize static objects in C++
        // C++ constructor for class Data
        // Start of main()
        // C++ main() function
        // C application function
        // C++ wrapper
        // C++ dynamic allocator
        // C++ constructor for class Data
        // C application function
        // C library function
        // C++ wrapper
        // C++ member function for class Data
        // C++ wrapper
        // C++ member function for class Data
        // C library function
        // C++ wrapper
        // C++ member function for class Data
        // C++ wrapper
        // C++ member function for class Data
        // C application function
        // C++ wrapper
        // C++ dynamic de-allocator
        // C++ destructor for class Data
        // End of main()
        // C++ destructor for class Data
    ~Data::Data()
```



Advanced Code Mix Scenarios

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

Advanced Code Mix Scenarios and Advisory



Advanced Code Mix Scenarios and Advisory

Tutorial T06

Partha Pratim
Das

Tutorial Recap

Objectives &
Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed
Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- The common code mix scenarios as described:
 - Are *simple to code* and *easy to debug*
 - Covers *most situations* in several projects
 - Are *stable*, *portable*, and *recommended*
- Beyond this, however, several other scenarios may need resolution from time to time:
 - Using **exceptions** in C++ code
 - ▷ C++ exception mechanism and rules about *destroying objects that go out of scope* are likely to be *violated* by a C `longjmp`, with *unpredictable results*
 - ▷ **ADVISORY:** Do not to use `longjmp` in programs that contain C++ code
 - ▷ *Stack unwinding* while the control passes through a C function, is *not portable or stable*
 - ▷ **ADVISORY:** Avoid exception path going through any C function
 - Manipulating objects in a **polymorphic hierarchy** should be carefully handled as
 - ▷ *C does not support dynamic dispatch*
 - ▷ **ADVISORY:** Use appropriate C++ wrappers to avoid getting into C's `if-else` type switch
 - Directly **accessing data members** in classes from C. This can be really tricky because
 - ▷ *Layout of objects in a hierarchy is not portable*
 - ▷ **ADVISORY:** Access data members only through appropriately designed C++ wrappers
 - Several project / software specific scenarios



Tutorial Summary

Tutorial T06

Partha Pratim Das

Tutorial Recap

Objectives & Outline

Mixing C & C++

Rules

Scenarios

C / C++ Mixed Project

Data.h

Data.cpp

Data_Wrap.cpp

App.c

main.cpp

makefile

Execution

Call Trace

Advanced Mix

Tutorial Summary

- We have learnt why is it often necessary to mix C and C++ codes in the same project
- We have explored the basic issues of mixing and learnt the ground rules
- In addition to the rules, we have four mechanisms to ease code mixing
 - Use `extern "C"` in C++ for all functions to be called from both C and C++
 - Guard `extern "C"` with `__cplusplus` guard for use with C
 - Provide `wrappers` for C++ data members, member functions, and overloaded functions for use with C
 - Incomplete `struct` type (with the same name as a C++ class) to allow pointers of C++ UDT objects in C
- We have also noted a few advanced mix scenarios and learnt the advisory of do's and don'ts